

Formula 1 Grand Prix Podium Finish Predictor

Using Machine Learning Methods to Predict Podium Finishes in Formula 1

By: Prithvi Rao
UCSB Spring 2026

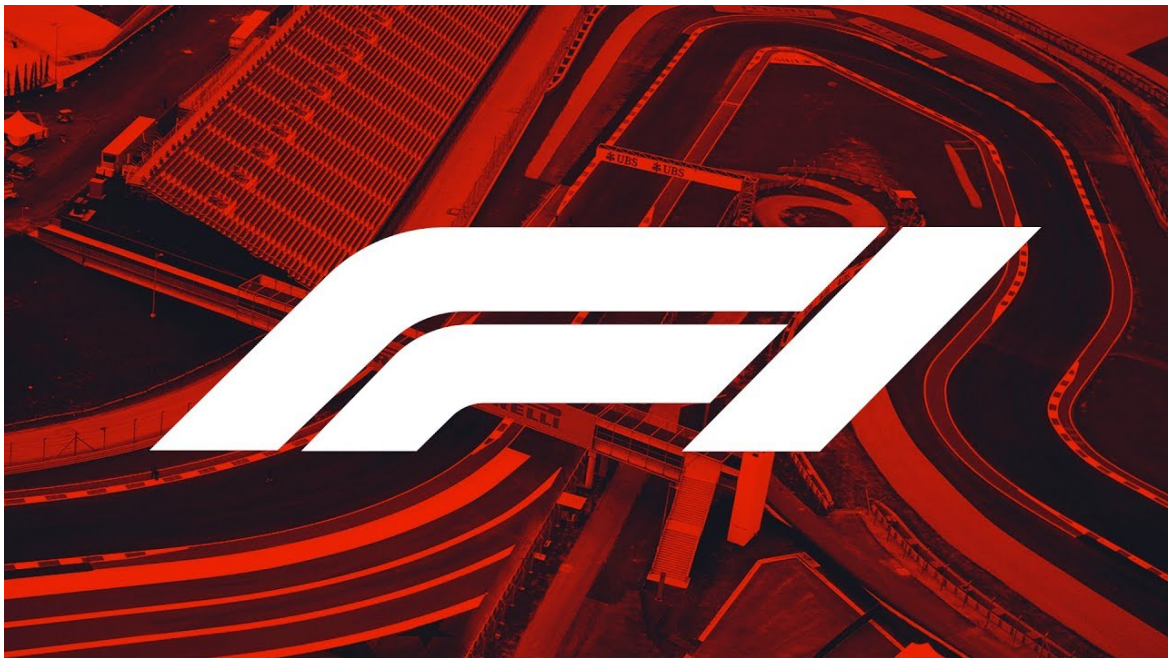


Table of Contents

1. Introduction
2. How Does Formula 1 Work?
3. Exploratory Data Analysis
 - Raw Data
 - Raw Data Summary
 - Merge Dataset Into One Main Dataset
 - Adding Additional Race Information To The Main Dataset
 - Checking Bahrain 2024 Grand Prix
 - Visual EDA
4. Before Race Data

- Creating Before-Race and Historical Performance Features
- Checks for Bahrain 2024 Grand Prix
- 5. Target Variable
- 6. Create the Modeling Dataset
 - Codebook
 - Handling Missing Data
 - Predictor Correlation Plot
- 7. Setting Up the Machine Learning Models
 - Setting Up Feature Sets for Modeling
 - Train/Test Split and Cross-Validation Setup
 - Define Models
- 8. Model Experiments
- 9. Model Tuning and Selection
 - Select Best Performing Model from Cross Validation
 - Tuning Grid
 - Tuning with Cross Validation
- 10. Results
- 11. Model Testing
- 12. Model Results
 - Model Accuracy by Constructor
 - Model Accuracy by Year
 - Variable Importance Plot
 - ROC-AUC of Model Over Time
 - Podium per Race
- 13. More Example Races
 - Monaco 2024 Grand Prix
 - Austrian 2022 Grand Prix
 - Dutch 2023 Grand Prix
- 14. Conclusion

Introduction

The goal of this project is to build a machine learning model that predicts whether a Formula 1 driver will finish on the podium, meaning in the top three, in a Grand Prix. This is therefore a binary classification problem because each driver result is classified as either a podium finish or a non-podium finish.

The data for this project comes from the [Formula 1 World Championship \(1950–2024\)](#) dataset on Kaggle. It includes race results, drivers, lap times, and other relevant race-related data that would be helpful to train the model to create

accurate predictions. This data set includes many CSV files, which can be joined with keys like `raceID` and `driverID`.

This dataset includes both categorical and numerical variables.

Numerical: grid position, finishing position, points, laps completed, qualifying rank, and standings points

Categorical: driver name, constructor/team, circuit, race name, nationality, and race status

How Does Formula 1 Work?

An F1 season is made up of multiple Grand Prix weekends. During this weekend, drivers participate in practice sessions, qualifying, and the main Grand Prix race.

There are three practice sessions where the driver and the team (constructor) prepare the car, but do not determine points or positions.

Qualifying determines a driver's starting position in the Grand Prix. It is broken up into three sessions:

- Q1 (18 minutes): All 20 drivers race. The 6 slowest drivers are eliminated and fill grid positions 16 through 20
- Q2 (15 minutes): The remaining 14 drivers compete. The 6 slowest drivers are eliminated to secure grid positions 11 through 15.
- Q3 (12 minutes): The final 10 drivers race to determine the top 10 starting spots for the Grand Prix.

The faster the driver, the lower their starting position is (pole position). Sometimes the final starting grid can differ from qualifying order because of penalties or car changes.

The Grand Prix is the main race. Drivers earn championship points based on their finishing position, and the top three finishers are considered podium finishers.

Exploratory Data Analysis

Before building the models, I explored and cleaned the datasets to better understand the data and which variables can be useful. In this section I will look at the overall structure of the data, check distributions and visualize important predictors like grid position, qualifying position, standings, and constructor performance. This helps identify patterns in the data and gives context for which

variables may be useful in predicting podium finishes.

Raw data

```
In [4]: circuits = pd.read_csv("../data/circuits.csv")
constructor_results = pd.read_csv("../data/constructor_results.csv")
constructor_standings = pd.read_csv("../data/constructor_standings.csv")
constructors = pd.read_csv("../data/constructors.csv")
driver_standings = pd.read_csv("../data/driver_standings.csv")
drivers = pd.read_csv("../data/drivers.csv")
lap_times = pd.read_csv("../data/lap_times.csv")
pit_stops = pd.read_csv("../data/pit_stops.csv")
qualifying = pd.read_csv("../data/qualifying.csv")
races = pd.read_csv("../data/races.csv")
results = pd.read_csv("../data/results.csv")
seasons = pd.read_csv("../data/seasons.csv")
sprint_results = pd.read_csv("../data/sprint_results.csv")
status = pd.read_csv("../data/status.csv")
```

The race data from the Kaggle dataset is split up into multiple tables with separate files for race results, qualifying results, drivers, constructors, circuits, standings, and other race information.

Raw Data summary

Let's iterate through all the tables and see what we are working with.

Out[5]:

	table	rows	columns	missing_values
0	circuits	77	9	0
1	constructor_results	12625	5	0
2	constructor_standings	13391	7	0
3	constructors	212	5	0
4	driver_standings	34863	7	0
5	drivers	861	9	0
6	lap_times	589081	6	0
7	pit_stops	11371	7	0
8	qualifying	10494	9	68
9	races	1125	18	0
10	results	26759	18	0
11	seasons	75	2	0
12	sprint_results	360	16	0
13	status	139	2	0

Majority of the tables seem to have no missing data. This is good! Qualifying is a very important indicator for driver performance and has 68 values missing, so we need to handle this before training the models.

But, how many podium and non-podium finishes are in the dataset?

Out[6]:

	finish_type	count
0	Not Podium	23362
1	Podium	3397

Wow! This table shows that this dataset is highly imbalanced. Most observations are non-podium finishes, with ~23k Not Podium and ~3.4k Podium. The model can be correct by predicting non-podium, which isn't useful to predict podium drivers. Therefore accuracy alone can be misleading, metrics like ROC-AUC, precision, recall, and F1-score are also needed.

Merge Dataset Into One Main Dataset

Since the original Formula 1 data is stored across multiple separate tables,

merging the relevant files into one master dataset would be beneficial for modeling and analysis. This step is necessary because the outcome variable comes from the race results table, while important predictors such as driver information, constructor information, circuit details, qualifying position, and standings come from different tables.

Each table contains key columns such as `raceId`, `driverId`, `constructorId`, `circuitId`, and `statusId`. I used these keys to merge the tables together, which ensures that each driver, race, constructor, circuit, and race status is matched correctly.

Out[7]:

	resultId	raceId	driverId	constructorId	number	grid	position	positionText
0	1	18	1	1	22	1	1	1
1	2	18	2	2	3	5	2	2
2	3	18	3	3	7	7	3	3
3	4	18	4	4	5	11	4	4
4	5	18	5	1	23	3	5	5

Adding Additional Race Information To The Main Dataset

After creating the first merged dataset, I added more race-related information from the other Formula 1 tables. These additional variables gave more context about each driver's race weekend. I added qualifying results, driver standings, constructor standings, constructor race results, pit stop summaries, lap time summaries, and sprint race results.

Qualifying data is one of the most important pre-race indicators of podium finish because it indicates how well that driver and their constructor perform on that specific track, so it was added. I also added driver and constructor standings to measure how strong each driver and team had been up to that point in the season. Pit stop and lap time data were summarized by driver and race using features such as number of pit stops, average pit stop time, total pit stop time,

average lap time, best lap time, and lap time variability. Sprint race data was also added for race weekends that included sprint events.

Out[16]:

	Data Added	New Features Created	Known Before Race?	Use in Final Model?
0	Qualifying Results	qualifying_position, q1, q2, q3	Yes	Yes
1	Driver Standings	driver_points_after_race, driver_standing_after_race, driver_wins_after_race	No - after race unless shifted	Only if converted to before-race values
2	Constructor Standings	constructor_points_after_race, constructor_standing_after_race, constructor_wins_after_race	No - after race unless shifted	Only if converted to before-race values
3	Constructor Race Results	constructor_points_race	No	No
4	Pit Stop Data	num_pit_stops, avg_pit_stop_ms, total_pit_stop_ms	No	No
5	Lap Time Data	avg_lap_ms, best_lap_ms, lap_time_std_ms	No	No
6	Sprint Results	sprint_grid, sprint_positionOrder, sprint_points	Yes, if sprint occurs before Grand Prix	Optional

This table summarizes the additional data merged into the main dataset. I will later discuss which variables are valid for pre-race prediction and which ones are excluded to avoid data leakage.

But first, let's check if the main dataset contains correct data.

Checking Bahrain 2024 Grand Prix



To make sure the main dataset was merged correctly, I checked the 2024 Bahrain Grand Prix as an example race.

Out[18]:

	racelid	year	round	circuitId	name	date	time
1101	1121	2024	1	3	Bahrain Grand Prix	2024-03-02	15:00:00 https://en.wikipedia.org/wiki/2024_Bahrain_Grand_Prix

Race Data

Out[20]:

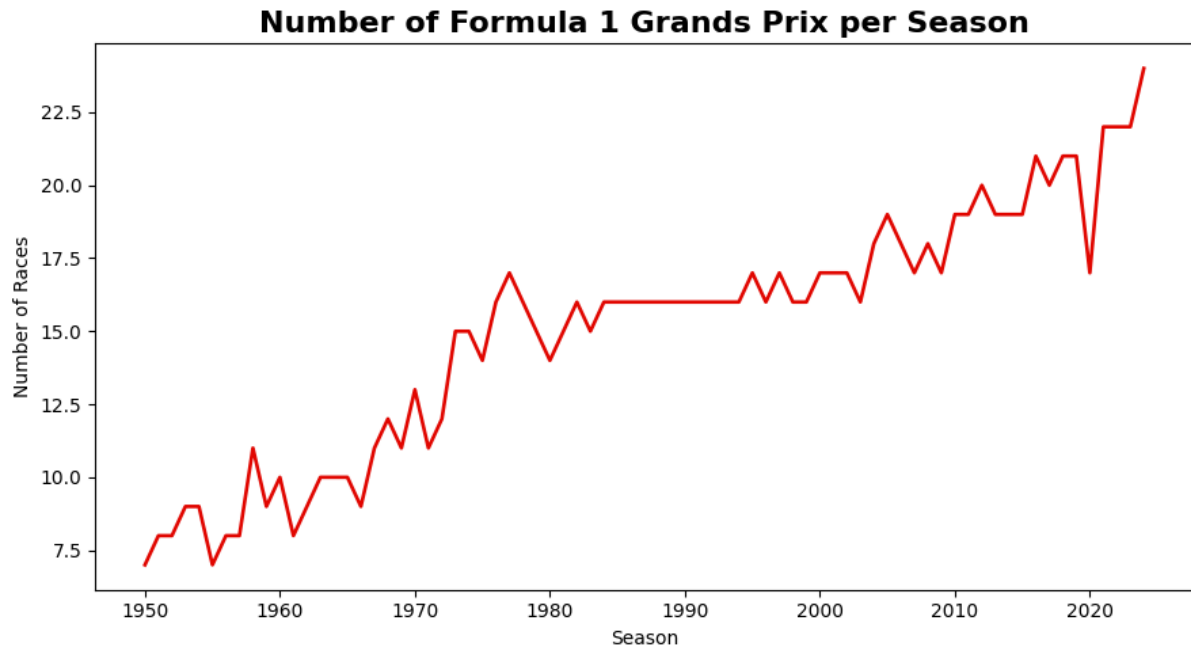
	racelid	year	round	circuit_name	date	driverId	forename	surnar
26280	1121	2024	1	Bahrain International Circuit	2024-03-02	830	Max	Verstapp
26281	1121	2024	1	Bahrain International Circuit	2024-03-02	815	Sergio	Pér
26282	1121	2024	1	Bahrain International Circuit	2024-03-02	832	Carlos	Sai
26283	1121	2024	1	Bahrain International Circuit	2024-03-02	844	Charles	Lecl
26284	1121	2024	1	Bahrain International Circuit	2024-03-02	847	George	Russ

Looks right! I verified this by cross-referencing the 2024 Bahrain Grand Prix rows

in the main dataset with the [official Formula 1 race results page](#) for the 2024 Bahrain Grand Prix.

Visual EDA

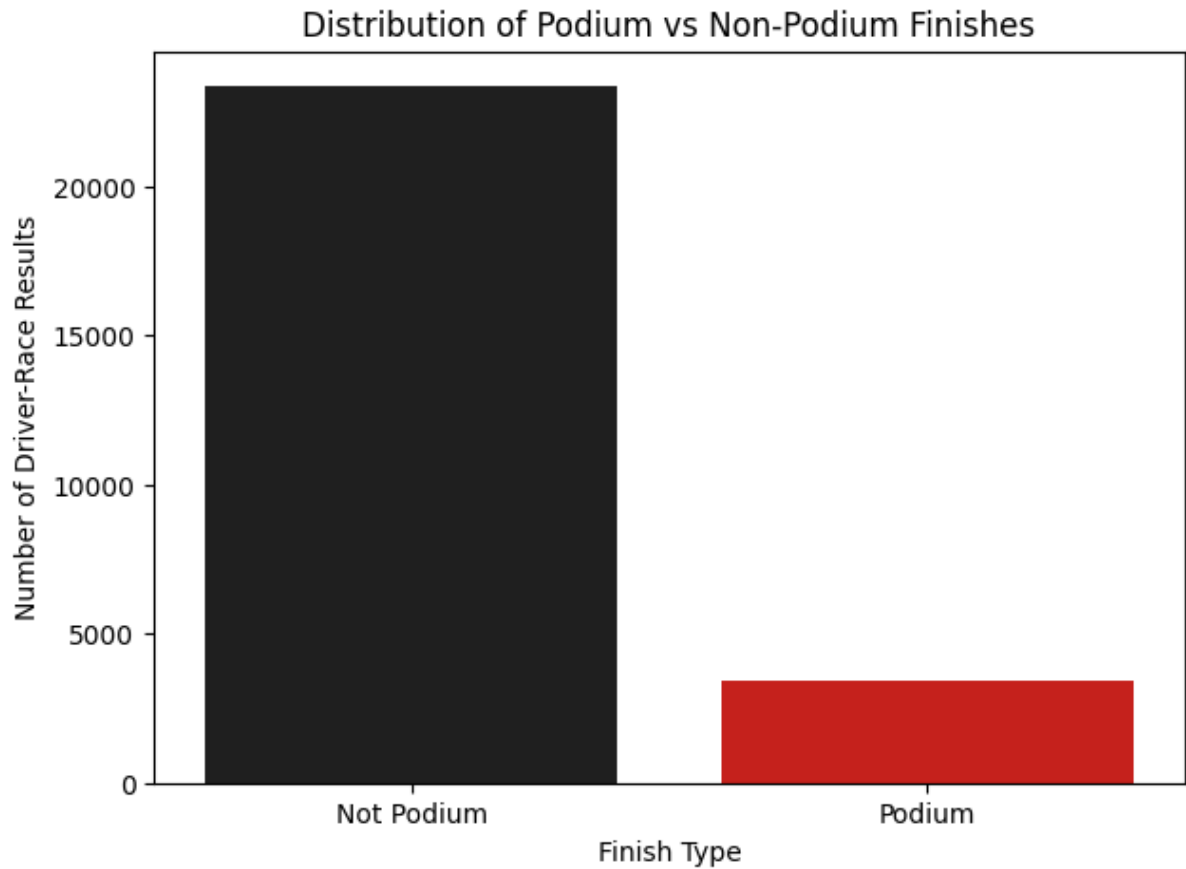
Number of Races Over Time



This plot shows how the Formula 1 calendar has changed over time. Earlier seasons had fewer races, while modern seasons include more Grands Prix. This matters because recent seasons contribute more observations to the dataset.

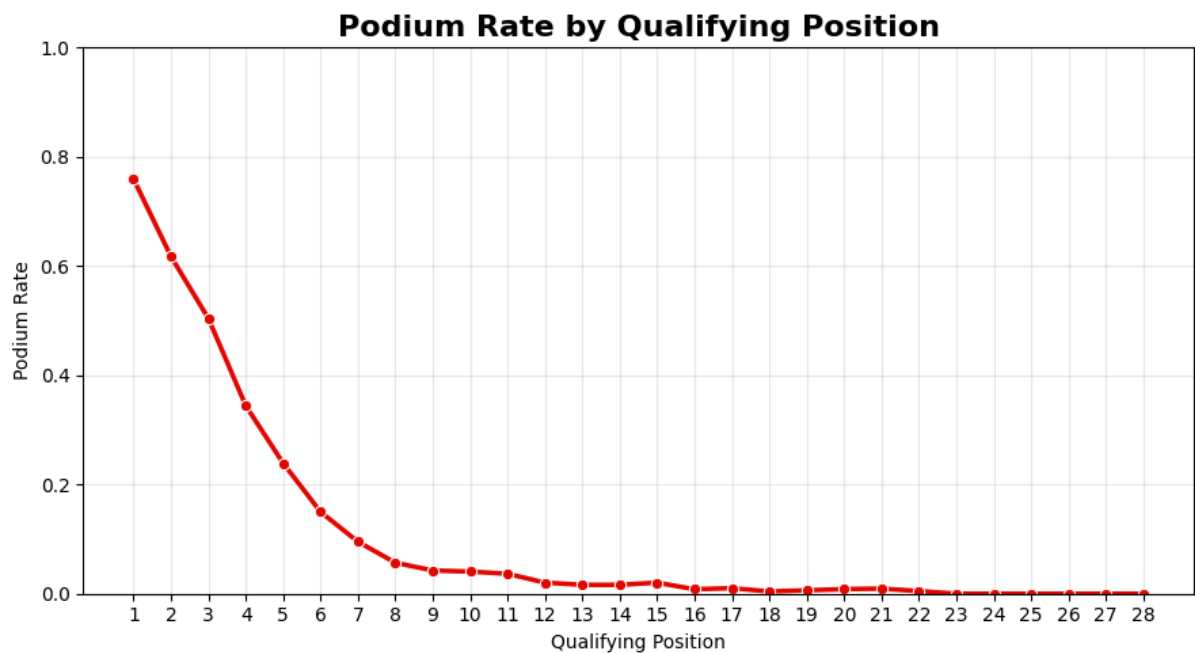
The increase in races also shows that older seasons may not reflect the same racing environment as modern seasons. The model may learn more useful patterns from more recent seasons where teams, qualifying structure, and overall competition are more similar to present day.

Podium vs non-podium distribution



This plot shows that podium finishes are much less common than non-podium finishes. This class imbalance is expected because only three drivers can finish on the podium. Therefore, accuracy alone is not enough to evaluate the models.

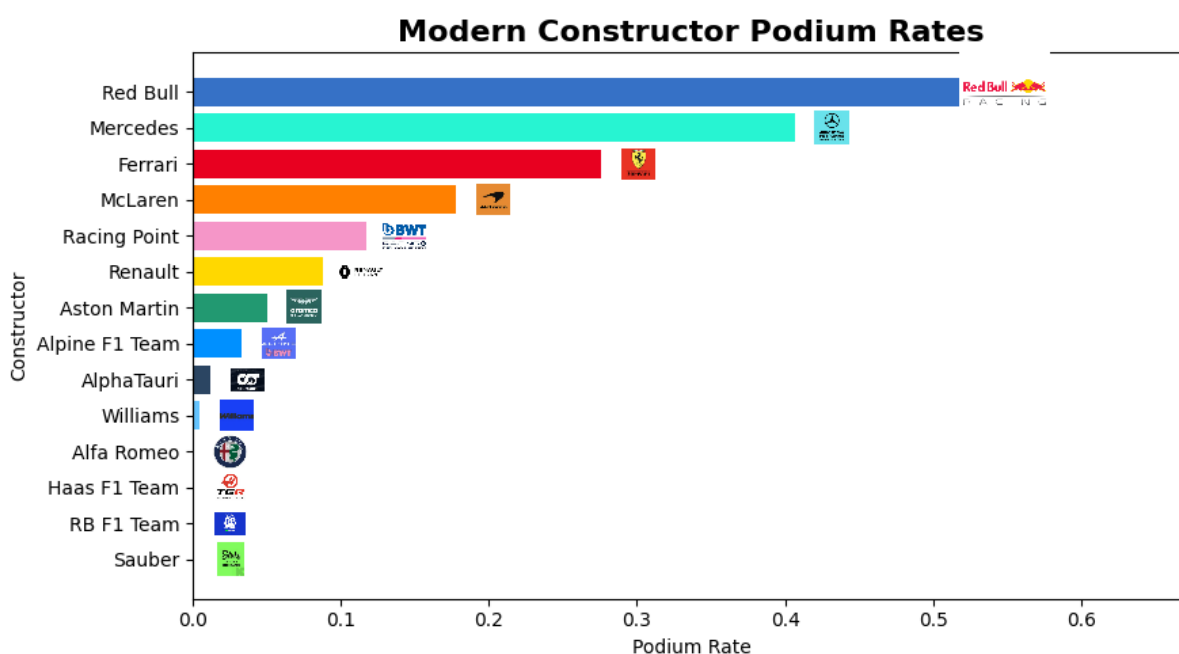
Podium Rate by Qualifying Position



This plot shows that starting position is strongly related to podium finishes. Drivers whose starting position is near the front of the grid have a much higher podium rate than drivers starting further back.

Since podium rate drops sharply after the first few starting positions, qualifying position should be used as one of the main predictors in the model. However, race outcomes can still be affected by strategy, reliability, penalties, crashes, weather, and team performance. Therefore qualifying position is important, but it should be combined with other pre-race variables rather than used alone.

Constructor Podium Rates



This plot compares podium rates across Formula 1 constructors from 2020 onwards. It indicates that certain teams tend to have much higher podium rates. Red Bull, Mercedes, and Ferrari seem to dominate the sport which could be a good indicator for predicting if a driver achieves a podium standing.

This is useful for the model because a driver's chance of finishing on the podium depends not only on the driver, but also on the strength of the car and team. Constructors with stronger cars are more likely to finish in the top three. Because of this, constructor-related variables are important predictors for podium classification as well.

Before Race Data

Some variables in this dataset are only known during or after the race, such as final position, points, pit stops, lap times, race status, fastest lap information, and race-level constructor points. These variables will not be used in the final pre-race prediction model because they would cause data leakage.

Creating Before-Race and Historical Performance Features

The goal of this project is to predict podium finishes before the race begins, so I needed to make sure the final model only used information that is available before each Grand Prix. Some variables, such as final position, race points, pit stops, lap times, race status, and fastest lap information, are only known during or after the race. These variables cannot be used because they would create data leakage.

To ensure the standings variables were valid for prediction, I shifted the driver and constructor standings by one race within each season. This created before-race features such as driver points before the race, driver standing before the race, driver wins before the race, constructor points before the race, constructor standing before the race, and constructor wins before the race. For the first race of each season, missing before-race values were filled with 0 for points and wins, and 99 for standings because no previous race information was available yet.

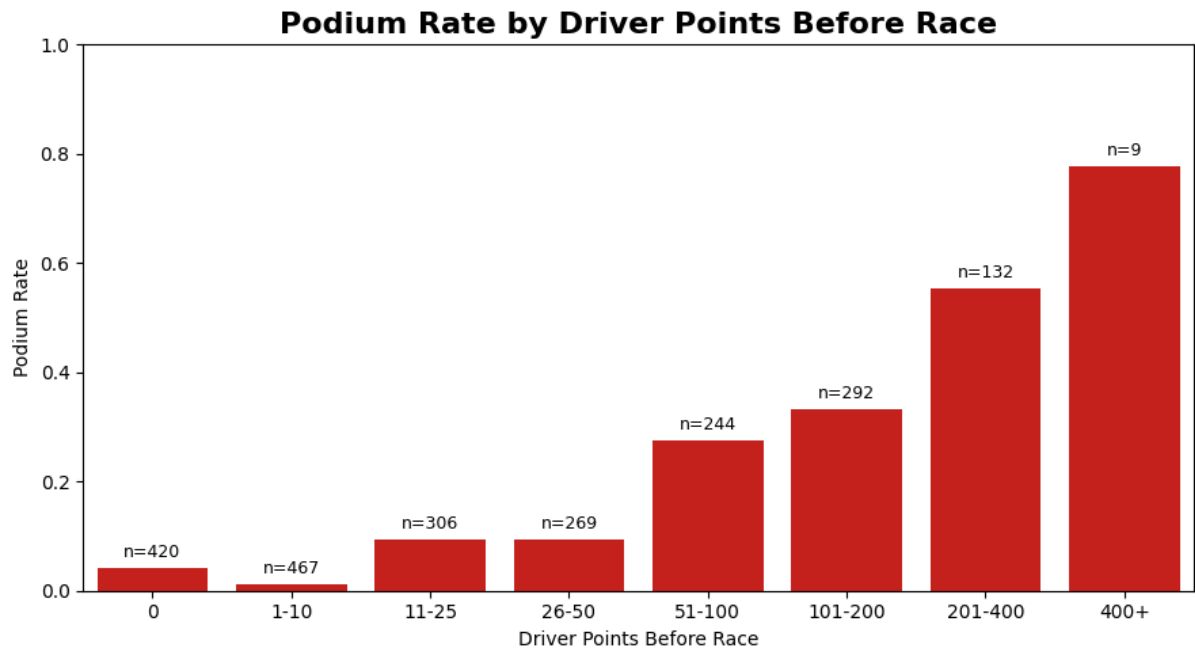
I also created historical performance features to represent each driver's recent form. I created driver-circuit history features based on the driver's last five races at the same circuit, including average finish, podium rate, and average points. I also created recent driver form features based on the driver's last five races, regardless of circuit. These features help the model capture whether a driver has been performing well recently or has historically performed well at a specific track.

Out [34] :

	Feature Group	Features Created	How It Was Created	Why It Is Useful
0	Driver before-race standings	driver_points_before_race, driver_standing_before_race, driver_wins_before_race	Shifted previous race standings within each season	Measures driver season strength before the race
1	Constructor before-race standings	constructor_points_before_race, constructor_standing_before_race, constructor_wins_before_race	Shifted previous race constructor standings within each season	Measures team season strength before the race
2	Driver circuit history	driver_circuit_avg_finish_last5, driver_circuit_podium_rate_last5, driver_circuit_avg_points_last5	Rolling average using previous 5 races at the same circuit	Captures driver history at a specific track
3	Driver recent form	driver_recent_avg_finish_last5, driver_recent_podium_rate_last5, driver_recent_avg_points_last5	Rolling average using previous 5 races overall	Captures recent driver performance

These are the main before-race and historical performance features created for the final modeling dataset. Each feature helps avoid data leakage by using only information from past races.

Driver points before race vs podium



This plot shows that drivers with more points before a race are much more likely to finish on the podium. Drivers with 0 or 1–10 points before a race have very low podium rates, while drivers with 201–400 points have a podium rate above 50%.

Overall, this supports using `driver_points_before_race` as a predictor because it captures current-season driver performance before the race begins. However, the sample sizes show that very high points are rare, so the model should not rely on this variable.

Checks for Bahrain 2024 Grand Prix

This table shows the before race data for the 2024 Bahrain Grand Prix.

Out [36]:

	racelId	year	round	circuit_name	date	driverId	forename	surname
24838	1121	2024	1	Bahrain International Circuit	2024-03-02	830	Max	Verstapp
23446	1121	2024	1	Bahrain International Circuit	2024-03-02	815	Sergio	Pérr
25087	1121	2024	1	Bahrain International Circuit	2024-03-02	832	Carlos	Sai
25948	1121	2024	1	Bahrain International Circuit	2024-03-02	844	Charles	Lecl
26225	1121	2024	1	Bahrain International Circuit	2024-03-02	847	George	Russ

Target Variable

```
In [37]: df["podium"] = (df["positionOrder"] <= 3).astype(int)

df["podium_place"] = np.where(
    df["positionOrder"] <= 3,
    df["positionOrder"],
    np.nan
)
```

```
In [38]: df["podium"].value_counts()
df["podium"].value_counts(normalize=True)
```

```
Out[38]: podium
0    0.873052
1    0.126948
Name: proportion, dtype: float64
```

This shows that the target variable is very imbalanced. About 87.3% of the observations are non-podium finishes, while only about 12.7% are podium finishes. This is why metrics like ROC-AUC and podium hit rate are more useful than accuracy for evaluating the results.

Create the modeling dataset

Here, I created a smaller modeling dataset that only keeps the variables used for

prediction. This step is pretty important because the main dataset contains many columns from the original Formula 1 tables, including identifiers, labels, race results, and other information that come from the race itself, not before. I made sure to only keep predictors that would be known before the Grand Prix begins. Variables such as finishing position and the podium label are still kept in the dataset, but they are treated as outcomes, not used as predictors.

Modeling Dataset Variables

Each row of the final modeling dataset includes information about the race, the driver, the constructor, the circuit, pre-race performance, historical performance, and whether the driver finished on the podium. The variables selected for the modeling dataset are:

raceId : Unique ID for each Formula 1 race.

driverId : Unique ID for each driver.

forename : Driver's first name.

surname : Driver's last name.

constructorId : Unique ID for each constructor/team.

constructor_name : Name of the constructor/team, such as Red Bull, Ferrari, Mercedes, or McLaren.

circuitId : Unique ID for each circuit.

circuit_name : Name of the race circuit, such as Silverstone Circuit or Circuit de Monaco.

year : The Formula 1 season year.

round : The race number within that season.

grid : The driver's starting grid position for the race.

qualifying_position : The driver's qualifying position before the race.

driver_points_before_race : The number of championship points the driver had before the race started.

driver_standing_before_race : The driver's championship standing before the race started.

driver_wins_before_race : The number of wins the driver had before the race started.

constructor_points_before_race : The number of championship points the constructor had before the race started.

constructor_standing_before_race : The constructor's championship standing before the race started.

constructor_wins_before_race : The number of wins the constructor had before the race started.

driver_circuit_avg_finish_last5 : The driver's average finishing position from their previous races at the same circuit, using up to the last five races.

driver_circuit_podium_rate_last5 : The driver's podium rate from their previous races at the same circuit, using up to the last five races.

driver_circuit_avg_points_last5 : The driver's average points from their previous races at the same circuit, using up to the last five races.

driver_recent_avg_finish_last5 : The driver's average finishing position over their previous five races overall.

driver_recent_podium_rate_last5 : The driver's podium rate over their previous five races overall.

driver_recent_avg_points_last5 : The driver's average points over their previous five races overall.

positionOrder : The driver's actual finishing position in the race. This is kept for evaluation.

podium : The response variable. It equals 1 if the driver finished in the top three and 0 otherwise.

Handling missing data

I also handled missing data before saving the final dataset. Qualifying position is one of the most important pre-race predictors and missing values mostly come from older races. I had to remove those rows instead of imputing those missing values. For historical driver variables, I filled the missing values with defaults because some drivers did not have enough previous races available. For example, missing recent average finish values were filled with 99, while missing podium

rate and points history values were filled with 0.

```
In [41]: modeling_data.isna().sum()
```

```
Out[41]:  raceId          0
         driverId       0
         forename       0
         surname        0
         constructorId   0
         constructor_name 0
         circuitId       0
         circuit_name    0
         year            0
         round           0
         grid            0
         qualifying_position 16275
         driver_points_before_race 0
         driver_standing_before_race 0
         driver_wins_before_race 0
         constructor_points_before_race 0
         constructor_standing_before_race 0
         constructor_wins_before_race 0
         driver_circuit_avg_finish_last5 0
         driver_circuit_podium_rate_last5 0
         driver_circuit_avg_points_last5 0
         driver_recent_avg_finish_last5 861
         driver_recent_podium_rate_last5 861
         driver_recent_avg_points_last5 861
         positionOrder   0
         podium          0
         dtype: int64
```

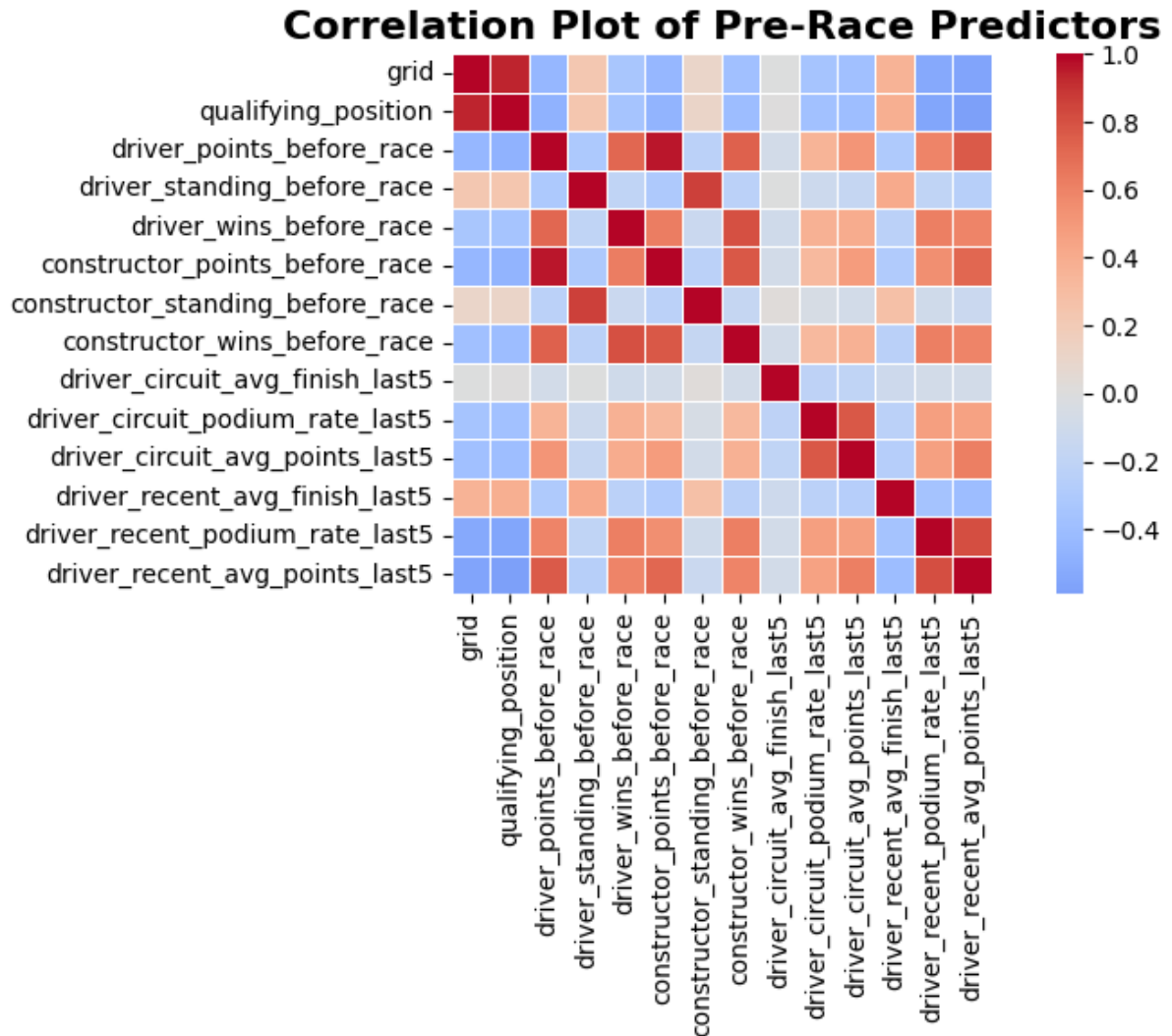
I drop rows with missing qualifying position data, which is a strong predictor. I also filled missing average finishes with 99 and podium rates as well as average points with 0.

```
In [42]: #drop rows missing qualifying position
modeling_data = modeling_data.dropna(subset=["qualifying_position"]).c

#fill missing recent driver history
modeling_data["driver_recent_avg_finish_last5"] = modeling_data["drive
modeling_data["driver_recent_podium_rate_last5"] = modeling_data["driv
modeling_data["driver_recent_avg_points_last5"] = modeling_data["drive

#fill missing circuit history
modeling_data["driver_circuit_avg_finish_last5"] = modeling_data["driv
modeling_data["driver_circuit_podium_rate_last5"] = modeling_data["dri
modeling_data["driver_circuit_avg_points_last5"] = modeling_data["driv
```

Predictor Correlation Plot



One of the clearest patterns I can see from this correlation plot is between `grid` and `qualifying_position`. This is to be expected because qualifying position usually indicates a driver's starting grid position in the Grand Prix race.

Nevertheless, this confirms that starting position is one of the most important pieces of pre-race information in Formula 1.

Another pattern I see is between the driver and constructor performance variables like driver points, constructor points, wins, or podium rates. This also makes sense because successful drivers usually race for stronger teams, and stronger teams tend to earn more points and podium finishes.

Overall, I think this plot confirms the feature choices to predict podium finishes. The predictors capture connected parts of Formula 1 performance, which gives the machine learning models useful information to learn from.

Setting up the Machine Learning Models

Setting Up Feature Sets for Modeling

The next step was to create feature sets so I could compare simple models against more detailed models. Instead of immediately using every variable, I broke the predictors into groups based on the type of information they represent. This makes the results easier to understand because I can see whether podium prediction is mostly driven by starting position, qualifying, standings, driver history, team strength, or a combination of them all.

grid_only : This is the baseline feature set and only uses starting grid position. I included this because grid position is already very important, so it gives a benchmark for comparison.

```
"grid_only":  
[  
    "grid"  
],
```

grid_and_qualifying : This feature set includes qualifying position as well as grid position. This set tests whether qualifying information improves performance in addition to grid position.

```
"grid_and_qualifying": [  
    "grid",  
    "qualifying_position"  
],
```

race_context : This feature set includes grid, qualifying position, year, round, and circuit ID. I used this to give more context about where and when the race took place, because different circuits can have different patterns.

```
"race_context": [  
    "grid",  
    "qualifying_position",  
    "year",  
    "round",  
    "circuitId"  
],
```

driver_constructor_circuit : This feature set includes driver ID, constructor ID, and circuit ID along with grid and qualifying position. I included this to test whether knowing driver, team, and track data helps the model learn

podium patterns better.

```
"driver_constructor_circuit": [  
    "grid",  
    "qualifying_position",  
    "driverId",  
    "constructorId",  
    "circuitId"  
],
```

season_standings : This feature set includes before-race driver and constructor points, standings, and wins. I used this group to capture how the driver and team had been during the current season.

```
"season_standings": [  
    "grid",  
    "qualifying_position",  
    "driver_points_before_race",  
    "driver_standing_before_race",  
    "driver_wins_before_race",  
    "constructor_points_before_race",  
    "constructor_standing_before_race",  
    "constructor_wins_before_race"  
],
```

driver_circuit_history : This feature set includes the driver's recent history at the same circuit, such as average finish, podium rate, and average points over previous races at that track. I used this to test whether looking at a driver's performance at a specific track can help the model.

```
"driver_circuit_history": [  
    "grid",  
    "qualifying_position",  
    "driver_circuit_avg_finish_last5",  
    "driver_circuit_podium_rate_last5",  
    "driver_circuit_avg_points_last5"  
],
```

driver_recent_form : This feature set includes the driver's last five races, regardless of circuit. I used this to capture recent momentum, which can be useful because drivers who have been performing well lately may be more likely to finish on the podium.

```
"driver_recent_form": [  
    "grid",  
    "qualifying_position",
```

```

        "driver_recent_avg_finish_last5",
        "driver_recent_podium_rate_last5",
        "driver_recent_avg_points_last5"
    ],

```

standings_and_circuit_history : This feature set combines current season standings with driver history at the same circuit. I used this to test whether season strength and track-specific history help the model.

```

"standings_and_circuit_history": [
    "grid",
    "qualifying_position",
    "driver_points_before_race",
    "driver_standing_before_race",
    "driver_wins_before_race",
    "constructor_points_before_race",
    "constructor_standing_before_race",
    "constructor_wins_before_race",
    "driver_circuit_avg_finish_last5",
    "driver_circuit_podium_rate_last5",
    "driver_circuit_avg_points_last5"
],

```

standings_and_recent_form : This feature set combines current season standings with recent driver form. I used this to see whether season-long performance and short-term momentum together improve podium prediction.

```

"standings_and_recent_form": [
    "grid",
    "qualifying_position",
    "driver_points_before_race",
    "driver_standing_before_race",
    "driver_wins_before_race",
    "constructor_points_before_race",
    "constructor_standing_before_race",
    "constructor_wins_before_race",
    "driver_recent_avg_finish_last5",
    "driver_recent_podium_rate_last5",
    "driver_recent_avg_points_last5"
],

```

all_valid_features : This is the full feature set that includes all valid pre-race predictors, including driver, constructor, circuit, race context, grid, qualifying, before-race standings, circuit history, and recent form. I used this to see how the models perform when they have access to all available pre-race information.

```

    "all_valid_features": [
        "driverId",
        "constructorId",
        "circuitId",
        "year",
        "round",
        "grid",
        "qualifying_position",
        "driver_points_before_race",
        "driver_standing_before_race",
        "driver_wins_before_race",
        "constructor_points_before_race",
        "constructor_standing_before_race",
        "constructor_wins_before_race",
        "driver_circuit_avg_finish_last5",
        "driver_circuit_podium_rate_last5",
        "driver_circuit_avg_points_last5",
        "driver_recent_avg_finish_last5",
        "driver_recent_podium_rate_last5",
        "driver_recent_avg_points_last5"
    ]

```

Train/Test Split and Cross-Validation Setup

Because Formula 1 data is time-based, I used a chronological split instead of a random split. I am using races from 2022 - 2024 as the final test set and earlier races for model training and validation.

In [46]: *#Chronological Train/Test Split*

```

train = modeling_data[modeling_data["year"] <= 2021].copy()
test = modeling_data[modeling_data["year"] >= 2022].copy()

print("Train shape:", train.shape)
print("Test shape:", test.shape)

print("\nTrain podium distribution: ", train["podium"].value_counts(normalized=True))
print("\nTest podium distribution: ", test["podium"].value_counts(normalized=True))

```

Train shape: (9125, 26)

Test shape: (1359, 26)

Train podium distribution: podium

0 0.859945

1 0.140055

Name: proportion, dtype: float64

Test podium distribution: podium

0 0.84989

1 0.15011

Name: proportion, dtype: float64

```
In [47]: cv = StratifiedKFold(  
        n_splits=5,  
        shuffle=True,  
        random_state=42  
    )
```

And in the training data, I used stratified 5-fold cross-validation. Stratification is useful because podium finishes are much less common than non-podium finishes, so each fold should keep a similar class balance.

Define Models:

Now it's time for the most exciting part of this project: building the machine learning models. I tested eight different classification models: Logistic Regression, Elastic Net Logistic Regression, LDA, QDA, KNN, Decision Tree, Random Forest, and Gradient Boosting. Using a wide variety of models was useful because I wanted to see how each model approaches the prediction problem differently. Models, like Logistic Regression and LDA, are simpler and easier to interpret, while models like Random Forest and Gradient Boosting are more flexible and can capture more complex patterns in the data.

Models Used

Logistic Regression : A baseline model for predicting podium vs. non-podium finishes.

Elastic Net Logistic Regression : A regularized logistic regression model used to control overfitting and handle correlated predictors.

LDA : A classic linear classification model used as another interpretable comparison method.

QDA : A more flexible version of LDA that can capture curved decision boundaries.

KNN : A distance-based model that predicts podium finishes by looking at similar driver-race situations.

Decision Tree : An interpretable tree-based model that can capture nonlinear patterns in the predictors.

Random Forest : A coordinated group of many decision trees used to improve performance and handle complex Formula 1 relationships.

Gradient Boosting : A boosted tree model that builds trees sequentially to improve prediction accuracy.

Model Experiments

Now that the models, feature sets, and the model data sets are defined, let's train them. I trained and evaluated each model using 5-fold cross-validation on the training dataset. For each feature set, I fit all eight models and calculated the average accuracy, precision, recall, F1-score, and ROC-AUC across the folds. This helped compare many model and feature set combinations to see which performs the best. I used ROC-AUC as the main comparison metric because the response variable is imbalanced, with many more non-podium finishes than podium finishes.

```
In [49]: model_results = []

In [50]: cv_results = []

scoring = {
    "accuracy": "accuracy",
    "precision": "precision",
    "recall": "recall",
    "f1": "f1",
    "roc_auc": "roc_auc"
}

for feature_set_name, features in feature_sets.items():

    X_train = train[features]
    y_train = train["podium"]

    for model_name, model in models.items():

        scores = cross_validate(
            model,
            X_train,
            y_train,
```

```

        cv=cv,
        scoring=scoring,
        n_jobs=-1
    )

    cv_results.append({
        "feature_set": feature_set_name,
        "model": model_name,
        "accuracy": scores["test_accuracy"].mean(),
        "precision": scores["test_precision"].mean(),
        "recall": scores["test_recall"].mean(),
        "f1": scores["test_f1"].mean(),
        "roc_auc": scores["test_roc_auc"].mean()
    })

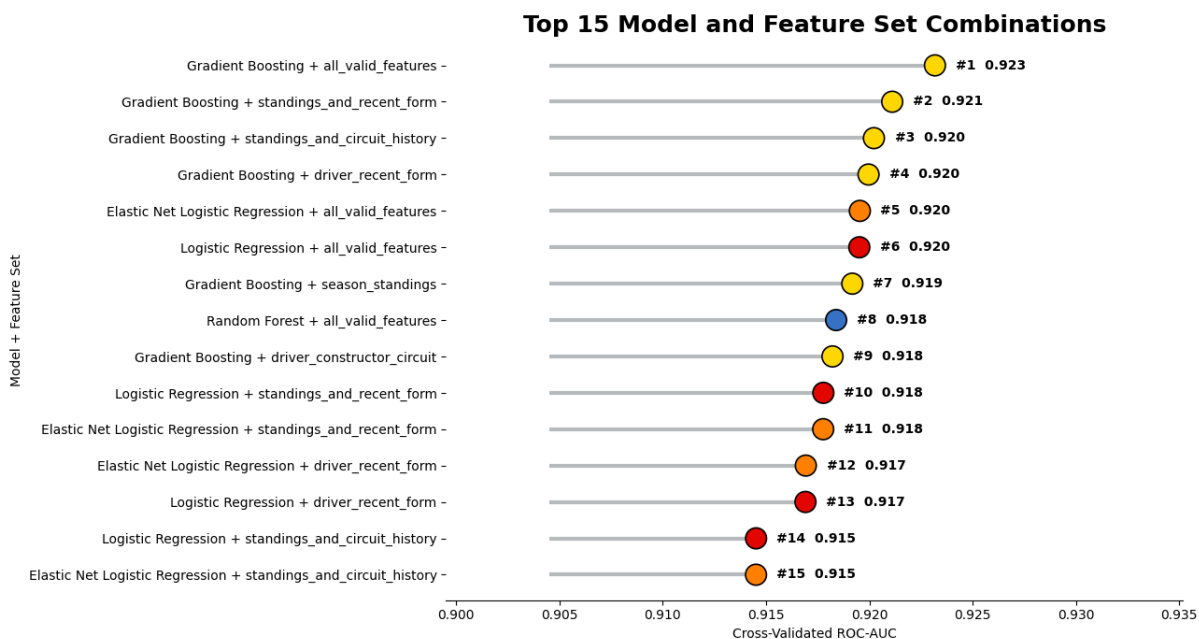
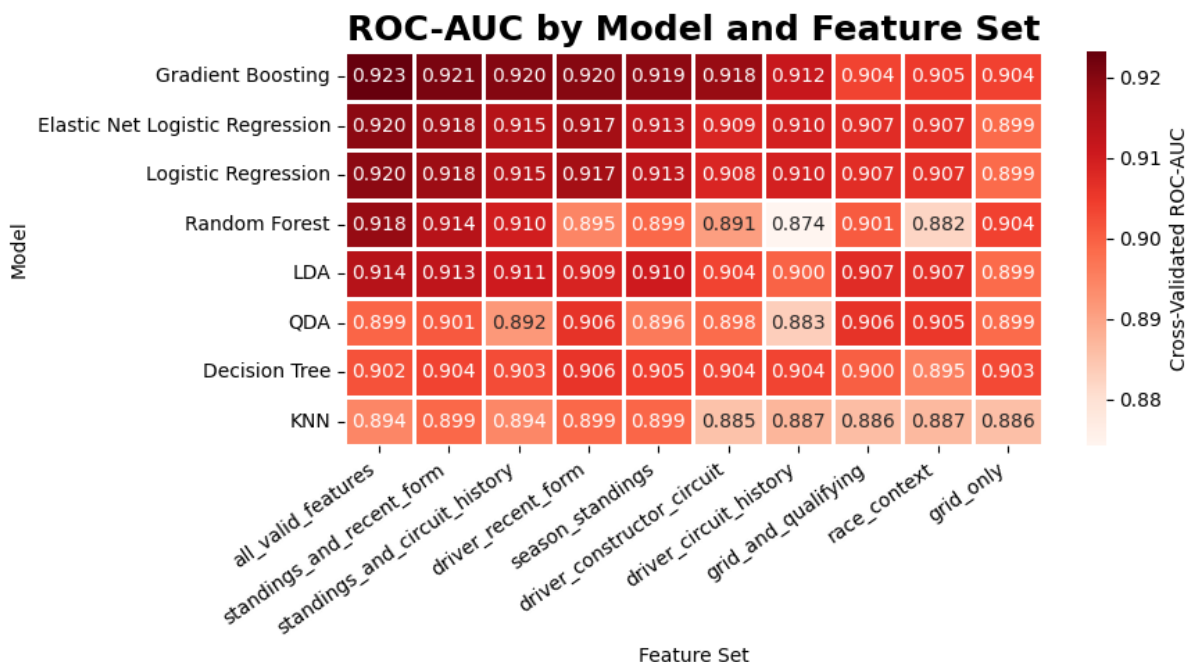
results_df = pd.DataFrame(cv_results)

results_df.sort_values("roc_auc", ascending=False).head(10)

```

Out[50]:

	feature_set	model	accuracy	precision	recall	
79	all_valid_features	Gradient Boosting	0.899068	0.665662	0.561057	0.60
71	standings_and_recent_form	Gradient Boosting	0.899288	0.669769	0.555561	0.60
63	standings_and_circuit_history	Gradient Boosting	0.898630	0.659889	0.570423	0.60
55	driver_recent_form	Gradient Boosting	0.900712	0.671244	0.570414	0.60
73	all_valid_features	Elastic Net Logistic Regression	0.897753	0.677432	0.515683	0.50
72	all_valid_features	Logistic Regression	0.897534	0.676079	0.515680	0.50
39	season_standings	Gradient Boosting	0.899836	0.664955	0.574354	0.60
78	all_valid_features	Random Forest	0.895781	0.676838	0.489816	0.50
31	driver_constructor_circuit	Gradient Boosting	0.898849	0.665961	0.557151	0.60
64	standings_and_recent_form	Logistic Regression	0.897425	0.680133	0.505512	0.50



The cross-validation results show that almost every model performed well, with ROC-AUC scores mostly around 0.89 to 0.92. We can see that the Gradient Boosting model had the strongest performance overall, especially when using the `all_valid_features` feature set, which had the highest ROC-AUC of about 0.923. This means that the model was able to use all the information from the full set of pre-race features to make accurate predictions.

Another thing I noticed is that the simpler models, like Logistic Regression and Elastic Net Logistic Regression, also performed very well. This shows podium prediction is already captured by strong pre-race variables like grid position,

qualifying position, and standings.

Overall, these results show that the chosen features were useful, but they also show that podium prediction has some strong patterns. Drivers who start near the front, qualify well, and enter the race with strong season performance are much more likely to finish on the podium.

Model Tuning and Selection

In this section I will take the features that the models performed the best on during cross-validation and tune them. The goal is to test different hyperparameter values and find the version of each model that performs the best on the training data.

The tuned models were KNN, Decision Tree, Random Forest, Gradient Boosting, and Elastic Net Logistic Regression. I can change the complexity of the model, so tuning them helps find a better balance between underfitting and overfitting.

For KNN, I tuned the number of neighbors and whether nearby observations should be weighted more heavily. For the Decision Tree, I tuned the maximum depth and minimum samples per leaf to control how complex the tree could become. For Random Forest, I tuned the number of trees, tree depth, minimum leaf size, and number of features considered at each split. For Gradient Boosting, I tuned the number of trees, learning rate, and tree depth. For Elastic Net Logistic Regression, I tuned the regularization strength and the balance between L1 and L2 regularization.

Select Best Performing Model from Cross Validation

```
In [54]: best_cv_model = results_df.sort_values("roc_auc", ascending=False).ilo  
  
best_model_name = best_cv_model["model"]  
best_feature_set_name = best_cv_model["feature_set"]  
best_features = feature_sets[best_feature_set_name]  
  
print("Best model from CV:", best_model_name)  
print("Best feature set:", best_feature_set_name)  
print("Best features:", best_features)
```

Best model from CV: Gradient Boosting

Best feature set: all_valid_features

Best features: ['driverId', 'constructorId', 'circuitId', 'year', 'round', 'grid', 'qualifying_position', 'driver_points_before_race', 'driver_standing_before_race', 'driver_wins_before_race', 'constructor_points_before_race', 'constructor_standing_before_race', 'constructor_wins_before_race', 'driver_circuit_avg_finish_last5', 'driver_circuit_podium_rate_last5', 'driver_circuit_avg_points_last5', 'driver_recent_avg_finish_last5', 'driver_recent_podium_rate_last5', 'driver_recent_avg_points_last5']

Tuning Grid

```
In [56]: # Tuning Grid
tuning_models = {

    "KNN": {
        "model": make_pipeline(
            StandardScaler(),
            KNeighborsClassifier()
        ),
        "params": {
            "kneighborsclassifier__n_neighbors": [5, 10, 15, 25, 35],
            "kneighborsclassifier__weights": ["uniform", "distance"]
        }
    },

    "Decision Tree": {
        "model": DecisionTreeClassifier(random_state=42),
        "params": {
            "max_depth": [2, 3, 5, 8, 12],
            "min_samples_leaf": [1, 5, 10, 25]
        }
    },

    "Random Forest": {
        "model": RandomForestClassifier(
            random_state=42,
            class_weight="balanced"
        ),
        "params": {
            "n_estimators": [100, 200],
            "max_depth": [3, 5, 8, None],
            "min_samples_leaf": [1, 5, 10],
            "max_features": ["sqrt", "log2"]
        }
    },

    "Gradient Boosting": {
        "model": GradientBoostingClassifier(random_state=42),
        "params": {
```

```

        "n_estimators": [50, 100, 200],
        "learning_rate": [0.01, 0.05, 0.10],
        "max_depth": [1, 2, 3]
    },
    "Elastic Net Logistic Regression": {
        "model": make_pipeline(
            StandardScaler(),
            LogisticRegression(
                solver="saga",
                max_iter=5000,
                random_state=42
            )
        ),
        "params": {
            "logisticregression__C": [0.01, 0.1, 1, 10],
            "logisticregression__l1_ratio": [0.1, 0.5, 0.9]
        }
    }
}

```

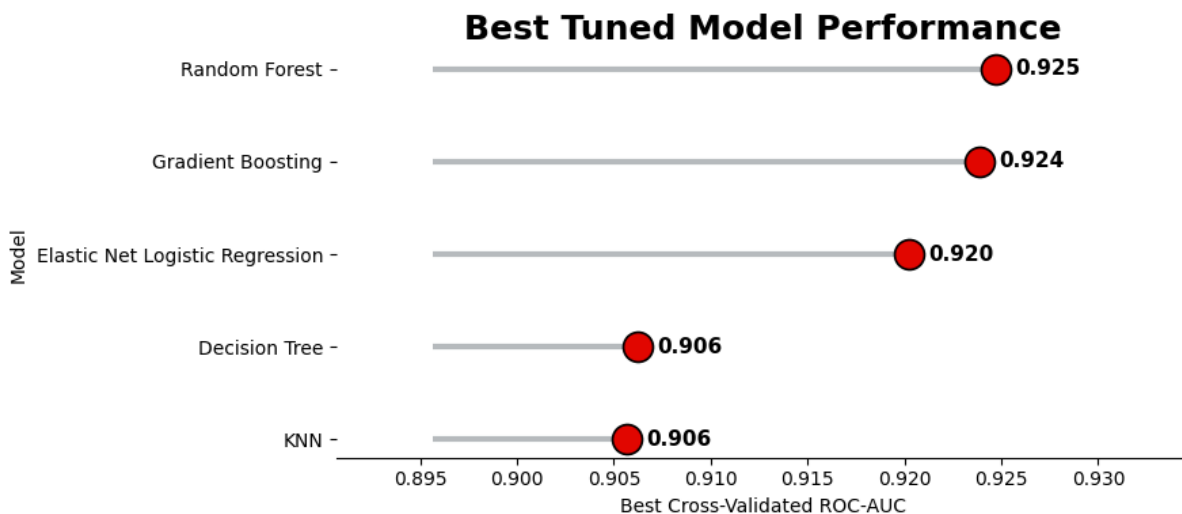
Tuning with Cross Validation

Out [57]:

	model	best_cv_roc_auc	best_params
2	Random Forest	0.924731	{'max_depth': 8, 'max_features': 'sqrt', 'min_samples_leaf': 10, 'n_estimators': 200}
3	Gradient Boosting	0.923873	{'learning_rate': 0.05, 'max_depth': 3, 'n_estimators': 200}
4	Elastic Net Logistic Regression	0.920264	{'logisticregression__C': 0.01, 'logisticregression__l1_ratio': 0.1}
1	Decision Tree	0.906206	{'max_depth': 5, 'min_samples_leaf': 25}
0	KNN	0.905668	{'kneighborsclassifier__n_neighbors': 35, 'kneighborsclassifier__weights': 'distance'}

This table shows the best tuned result and parameters of each model. Random Forest had the highest ROC-AUC at about 0.925, with Gradient Boosting very close at about 0.924.

Results



The tuning results show that the best-performing tuned model was Random Forest, with a cross-validated ROC-AUC of about 0.925. This is interesting because before tuning, Gradient Boosting had the highest ROC-AUC score. After tuning Gradient Boosting was extremely close behind at about 0.924. This shows that tree-based methods are better at capturing useful patterns in the Formula 1 data. Both are strong choices, but I will be using the Random Forest model for final testing.

Model Testing

The best tuned model was Random Forest, with a cross-validated ROC-AUC of about 0.925. The best parameters were `max_depth = 8`, `max_features = sqrt`, `min_samples_leaf = 10`, and `n_estimators = 200`. Now, I fit it on the full training set and evaluated it on the testing set.

On the training set, the Random Forest model has a ROC-AUC of about 0.925, let's see how it performs on the testing set.

```
In [60]: best_tuned_row = tuned_results_df.sort_values("best_cv_roc_auc", ascending=False)
best_tuned_model_name = best_tuned_row["model"]
best_tuned_model = best_tuned_models[best_tuned_model_name]

print("Best tuned model:", best_tuned_model_name)
print("Best CV ROC-AUC:", best_tuned_row["best_cv_roc_auc"])
print("Best parameters:", best_tuned_row["best_params"])
```

```
Best tuned model: Random Forest
Best CV ROC-AUC: 0.9247306586466169
Best parameters: {'max_depth': 8, 'max_features': 'sqrt', 'min_samples_leaf': 10, 'n_estimators': 200}
```

```

In [61]: X_train = train[best_features]
Y_train = train["podium"]

X_test = test[best_features]
Y_test = test["podium"]

#fit model on full training set
best_tuned_model.fit(X_train, Y_train)

#predictions
Y_pred = best_tuned_model.predict(X_test)
Y_prob = best_tuned_model.predict_proba(X_test)[:, 1]

#test metrics
test_results = pd.DataFrame({
    "model": [best_tuned_model_name],
    "feature_set": [best_feature_set_name],
    "test_accuracy": [accuracy_score(Y_test, Y_pred)],
    "test_error": [1 - accuracy_score(Y_test, Y_pred)],
    "test_precision": [precision_score(Y_test, Y_pred)],
    "test_recall": [recall_score(Y_test, Y_pred)],
    "test_f1": [f1_score(Y_test, Y_pred)],
    "test_roc_auc": [roc_auc_score(Y_test, Y_prob)]
})

test_results

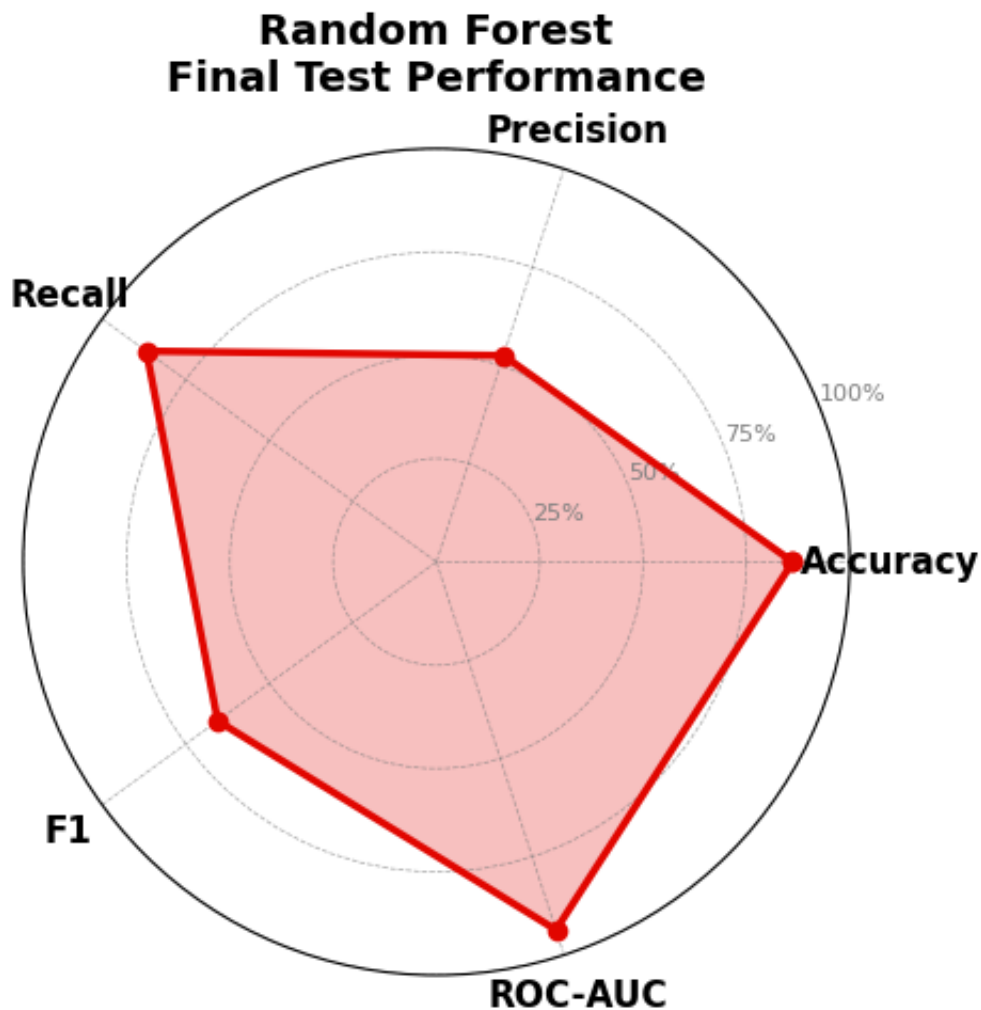
```

```

Out[61]:

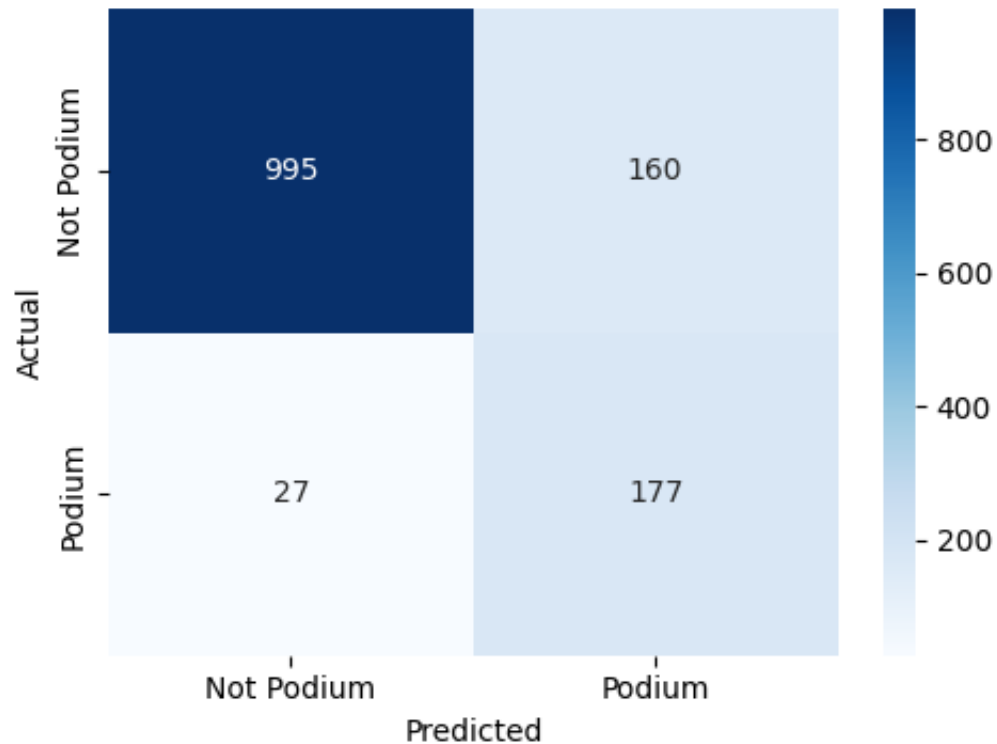
```

	model	feature_set	test_accuracy	test_error	test_precision	test_rec
0	Random Forest	all_valid_features	0.862399	0.137601	0.525223	0.8676



On the test set we can see that the Random Forest model had an ROC-AUC of about 0.936, and an accuracy of 0.862. Meaning that the model did pretty well at separating likely podium finishers from non-podium finishers. Accuracy is not that important as an evaluation metric because imbalance in the data is caused by a majority of the drivers not finishing on the podium. A recall score of 0.868, means that the model was able to identify a large portion of the actual podium finishers. And the precision score of 0.525 means that the model did end up predicting a driver would end up on the podium, but did not actually finish in the top three.

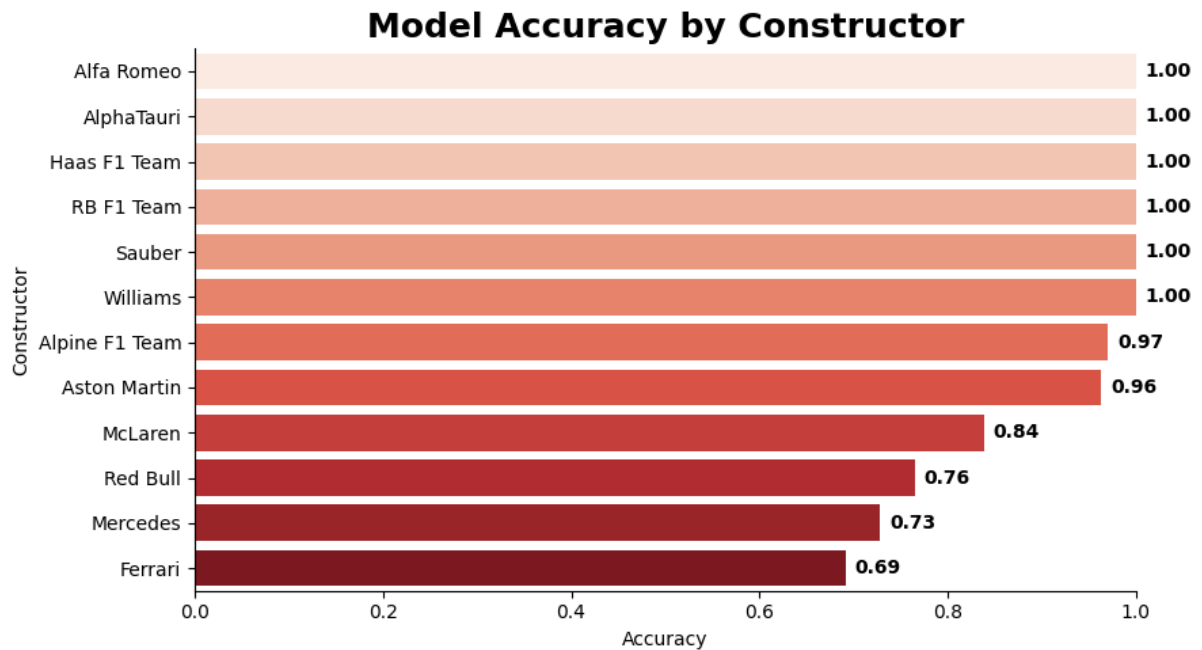
Confusion Matrix for Final Tuned Model



Overall, for the test data, we can see that the model correctly predicted 995 non-podium finishes and 177 podium finishes. It also predicted 160 drivers as podium finishers who did not actually finish in the top three. This explains why the precision score is lower than the recall score.

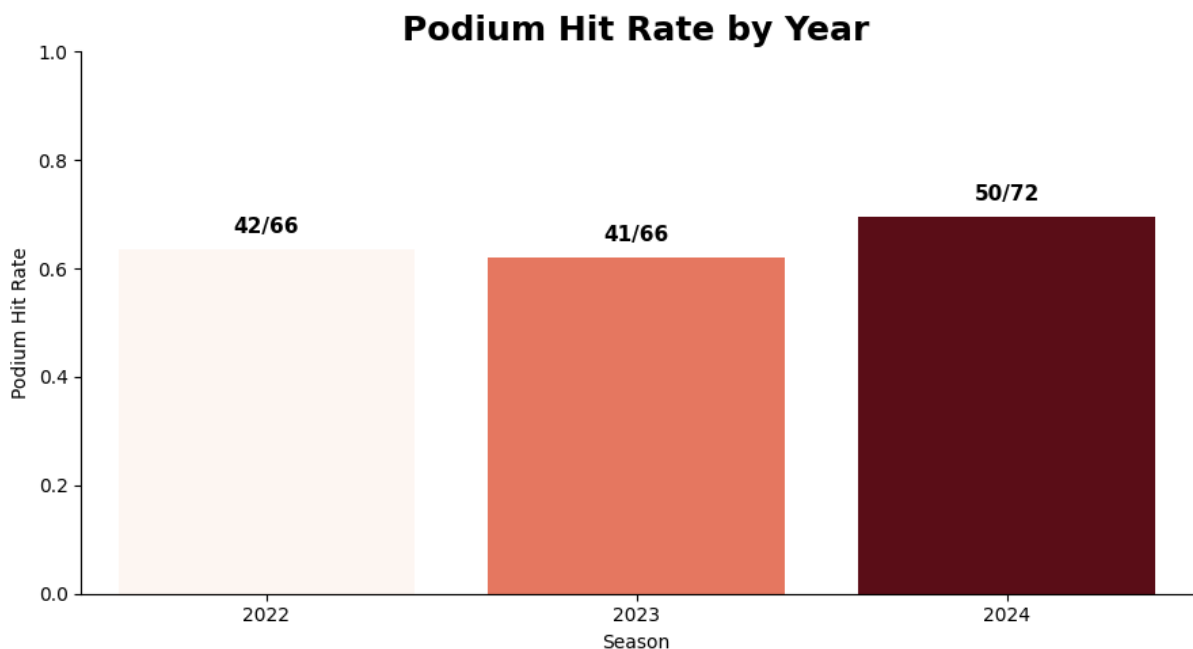
Model Results

Model Accuracy by Constructor



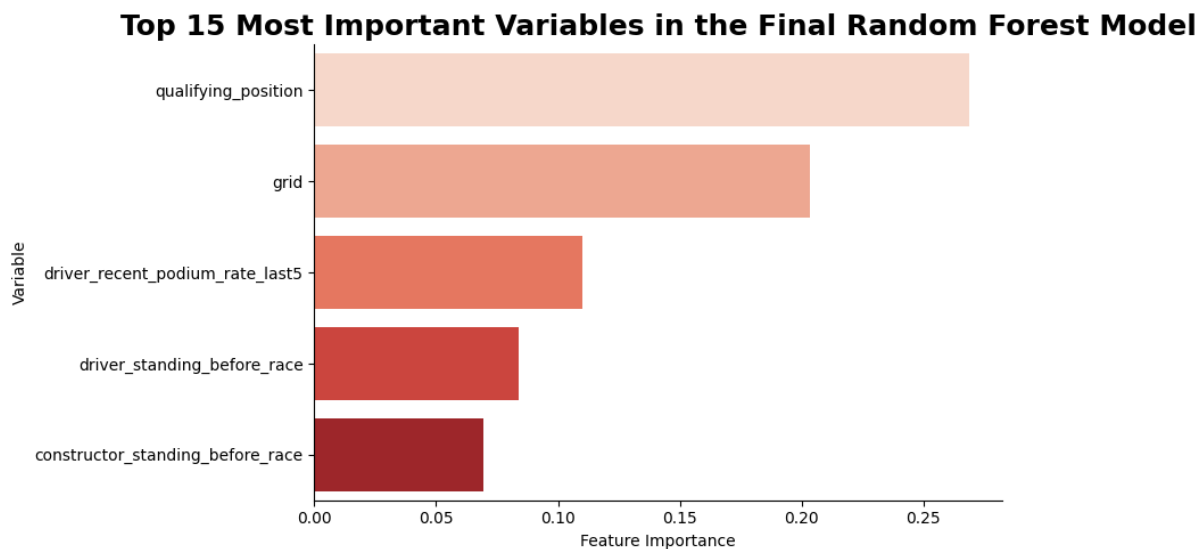
This plot shows that the model had very high accuracy for many lower and midfield constructors. This makes sense because accuracy also measures how well the model predicted “not podium”. Since these teams rarely finished on the podium, the model is correct when predicting them as non-podium finishers. Accuracy was lower for top-tier teams like Ferrari, Mercedes, and Red Bull because they were more likely to fight for podium spots, making their results harder to predict.

Model Accuracy by Year



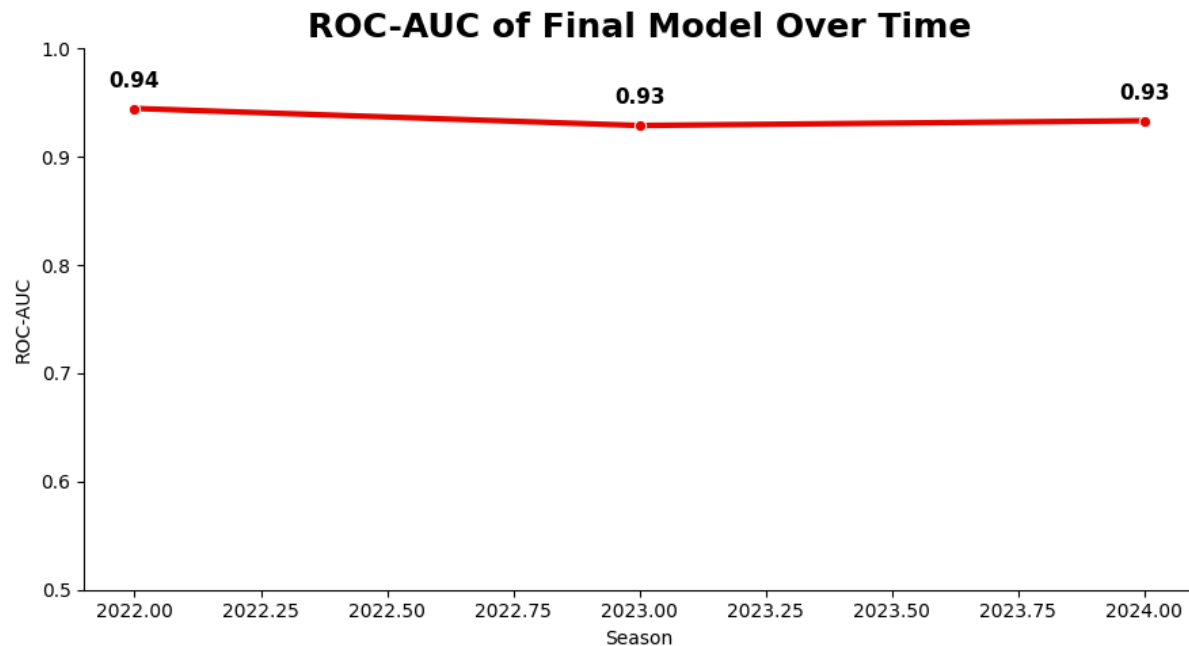
This plot shows that the model's podium hit rate was fairly consistent across the test years, but it performed best in 2024.

Variable Importance Plot



We can clearly see that qualifying position and starting grid position were the two most important predictors in the final Random Forest model. This makes sense because drivers who perform better in qualifying and start near the front are more likely to finish on the podium. It is also clear that driver and constructor historical data does play a role in a driver's predictions.

ROC-AUC of Model Over Time



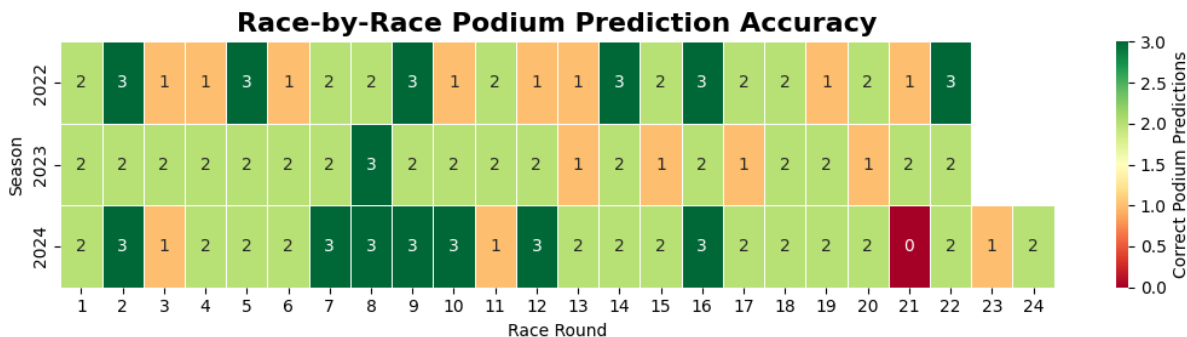
This plot shows that the model's ROC-AUC stayed very consistent from 2022 to 2024. The scores stayed around 0.93–0.94, which means the model performed well across the test seasons.

Podium per Race

```
In [71]: correct_podium_predictions = (
    (test_predictions["podium"] == 1) &
    (test_predictions["predicted_podium_top3"] == 1)
).sum()

total_actual_podiums = test_predictions["podium"].sum()

podium_hit_rate = correct_podium_predictions / total_actual_podiums
```

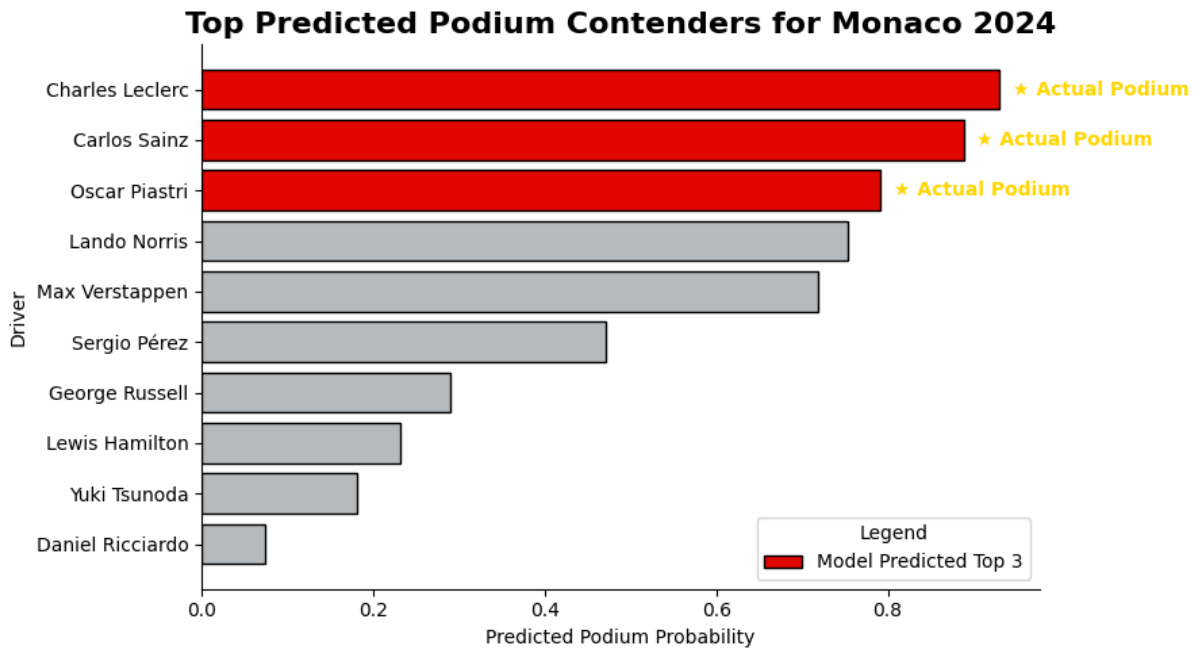


This heatmap shows how many podium finishers the model correctly predicted in each race in the test data set. In most of the races, the model usually predicted 2 or 3 of the actual podium drivers correctly. But in one of the races, 2024 Race 21, the model did not correctly predict any of the podium drivers. Why is that?

Let's take a deeper dive into that race. This race was the 2024 São Paulo Grand Prix, where Max Verstappen had a huge comeback from 17th on the starting grid to take first. In addition, the track conditions that day were very difficult because the race was affected by heavy rain and wet conditions. Verstappen's comeback, wet conditions, as well as exceptional performance from both Alpine drivers, the midfield underdog of Formula 1, made this race difficult to predict. The model relies on only information from before the race, so it was not able to predict the podium given these unusual circumstances. This highlights one of the main limitations of the model. Complex races with wet conditions, unusual strategy calls, safety cars, or unexpected incidents are much harder to predict using pre-race information.

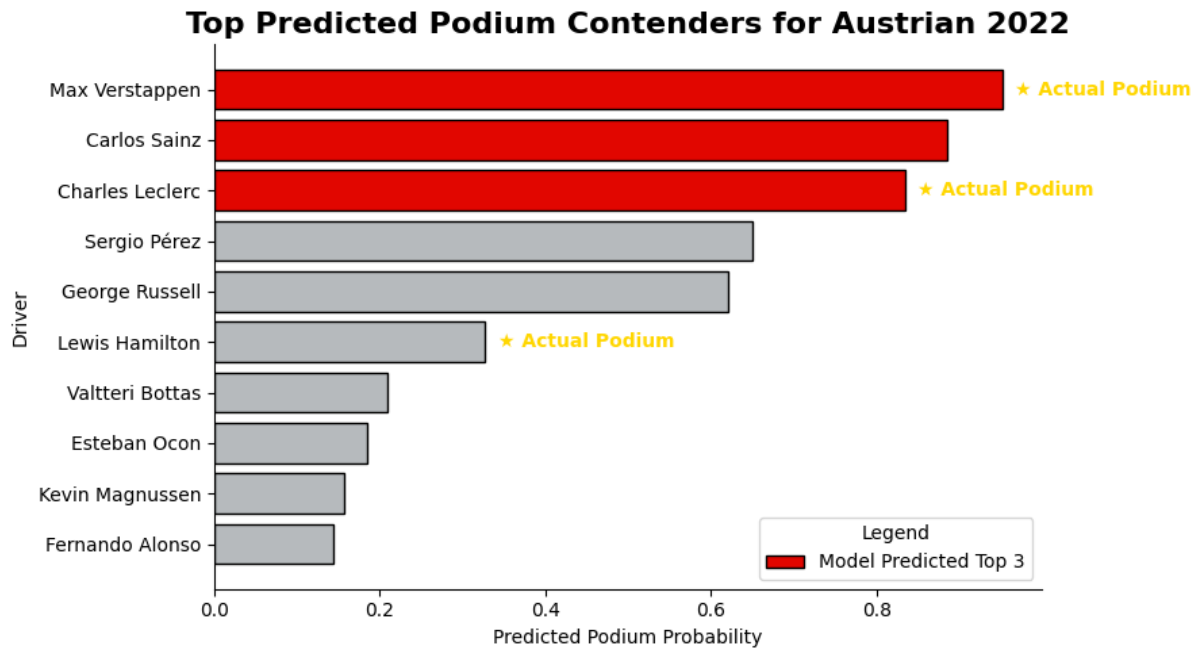
More Example Races

Monaco 2024 Grand Prix



This Monaco 2024 example shows the model working exactly as intended. The model's predicted top three drivers were Charles Leclerc, Carlos Sainz, and Oscar Piastri, and all three of them actually finished on the podium. This is a strong example of the model correctly ranking the main podium drivers with before race data.

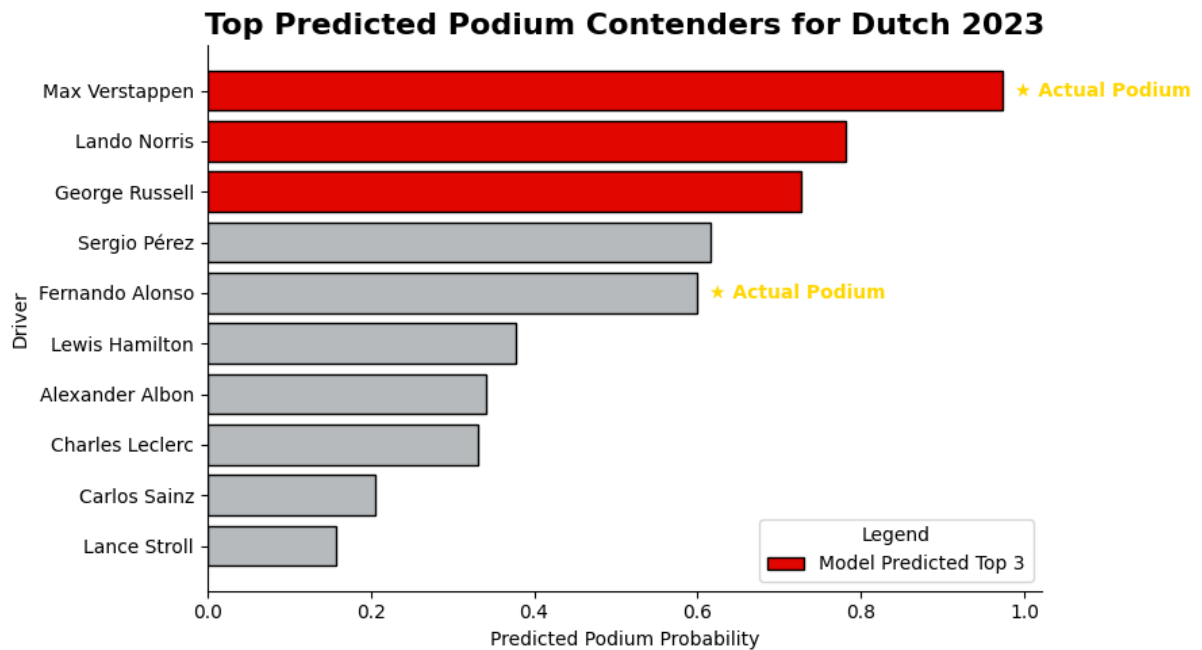
Austrian 2022 Grand Prix



The model got 2 out of 3 podium finishers correct for the 2022 Austrian Grand Prix. The miss makes sense because Carlos Sainz was a strong podium contender, but he was forced to stop racing because his car caught on fire. Yikes! The model only uses pre-race information, so it was impossible to predict an issue like that, which allowed Lewis Hamilton to take the final podium spot. This is a good example of a race-day event that the model could not predict using only pre-race data.

Dutch 2023 Grand Prix

```
In [78]: plot_race_predictions(2023, 13, "Dutch 2023")
```



This plot does not show the third podium finisher, so I wanted to check who that was ...

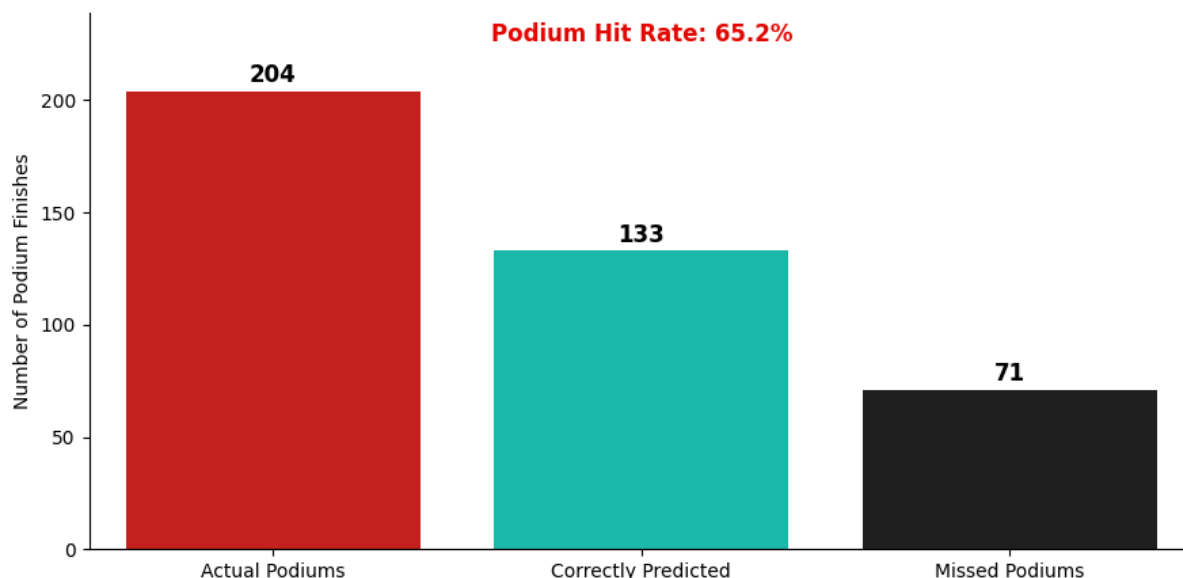
Out[79]:

	raceld	year	round	forename	surname	constructor_name	position
26086	1111	2023	13	Max	Verstappen	Red Bull	
26081	1111	2023	13	Fernando	Alonso	Aston Martin	
26090	1111	2023	13	Pierre	Gasly	Alpine F1 Team	

The reason we didn't see Pierre Gasly on the plot was because he only had a 0.058660 chance of ending up on the podium. In the race, Pierre Gasly benefited from wet conditions, good strategy calls, and Sergio Pérez receiving a penalty, which helped move him onto the podium. This means that the model can struggle when predicting for underdog drivers because of events that were not captured in the pre-race data.

Conclusion

Final Model Podium Prediction Summary



Conclusion

After testing and analysis, the best final model for predicting Formula 1 podium finishes ended up being a tuned Random Forest model. This model performed really well, especially since Formula 1 is a very complex sport where there are a lot of factors that affect a driver's performance. The Random Forest model had a ROC-AUC of about 0.936, meaning it was very strong at ranking actual podium finishers apart from non-podium finishers. It correctly predicted 133 out of 204 actual podium finishers, giving it a podium hit rate of about 65.2%. Considering one crash, strategy call, penalty, or wet track condition can completely change the trajectory of a race, I was impressed with the results.

One thing I noticed while exploring different models and their performance was that ensemble models were strongest overall. Gradient Boosting performed the best before tuning, but after tuning, Random Forest passed it by a small margin and became the final selected model. Formula 1 performance is based on a combination of qualifying, grid position, driver form, constructor strength, and season standings, making ensemble models a strong choice for capturing complex relationships between multiple predictors. The variable importance plot supports this because qualifying position and grid position were the most important predictors. This project also showed me why accuracy alone sometimes can be misleading. Accuracy also takes into account the drivers who did not finish on the podium, so a model can look stronger than it actually is at identifying podium finishers. This is why ROC-AUC and podium hit rate were more useful for understanding how well the model actually worked.

For future improvements, I would like to include race-day variables like weather, tire strategy, safety cars, penalties, reliability issues, and track conditions. The 2024 São Paulo Grand Prix was a great example where these variables could be useful because the race was chaotic, wet, and had a very unexpected podium, which made it difficult for a pre-race model to predict.

Overall, this project really gave me a strong foundation in machine learning and sports analytics, and I am excited to keep building on it to become a real-time Formula 1 analytics tool. It was also really amazing to see how Formula 1 performance can be modeled, and it raises the question of where else data can be applied to predict the future.

Sources

Data Source

The data used in this project comes from the Formula 1 World Championship dataset on Kaggle.

Citation: Kaggle. [Formula 1 World Championship \(1950–2024\)](#)

Images

[team logos](#)

[Bahrain 2024 track image](#)

[drivers image](#)

[F1 logo](#)

Plots and Image Injection

I used ChatGPT to help embed some images into the HTML report, and helped me create some visualization and plotting code.